

REFERENCE

Action-Directive System – Reference Manual

Revision: 2 **Last modified:** 2026-07-14T22:05:00Z **Authority:** constitution §11.4.140 (universal action-prefix system) + §11.4.202 (reporting directives) + §11.4.164 (post-update auto-propagation) **Maintainer:** constitution submodule (inherited BY REFERENCE per §11.4.28(B) / §11.4.177 — never copied into a consuming project) **Scope:** the complete reference for the registry, the grammar, the plugin, the per-agent commands, the skills, and every script that implements them.

Companion documents: [USER_GUIDE.md](#) · [ADMIN_MANUAL.md](#) · [QUICKSTART_REPORT.md](#) · [QUICKSTART_INSTALL.md](#)

1

. What the system is

A prompt whose **first non-blank line** starts with a registered UPPERCASE action token is not an ordinary prompt. The token is **replaced** by that action's registered `expansion` text, the action's `rules` apply, and the remainder of the prompt is the actual task, executed under the expanded instruction.

`actions/registry.yaml` is the **single source of truth**, and it is **DATA**: adding a directive is a registry row — never a code change in any consumer.

The system has two independent layers. Both read the same registry.

Layer	Mechanism	Where
LAYER 1 — recognition	The agent reads the §11.4.140 block in its context carrier (<code>CLAUDE.md</code> / <code>AGENTS.md</code> / <code>QWEN.md</code> / <code>GEMINI.md</code>) and expands the prefix itself. Works in any agent, including ones with no hook or command support.	<code>actions/</code> <code>recognition_instruction.md</code> (the canonical carrier block)
LAYER 2 — mechanical	(a) A Claude Code <code>UserPromptSubmit</code> hook rewrites/augments the prompt deterministically before the model sees it. (b) Generated slash commands per agent. Both survive a model recall lapse (the §11.4.109 anti-forgetting pattern).	<code>scripts/hooks/</code> <code>action_prefix_expand.sh</code> , <code>plugins/helix/commands/</code> , <code>actions/generated/<agent>/</code>

Honest boundary (§11.4.6). LAYER 2's prompt-expansion hook exists only for Claude Code. Gemini CLI / Qwen Code / Codex CLI get LAYER 1 (the carrier block) plus generated slash commands — they have **no** pre-submit hook, so a free-form `BUG: ...` prompt on those agents is expanded by the model reading its carrier block, not by a hook.

. File layout

Path	Kind	Purpose
actions/registry.yaml	DATA — source of truth	Grammar constants, the registered actions: , the subsystems: catalogue.
actions/recognition_instruction.md	doc	The verbatim LAYER-1 block embedded into every agent context carrier.
actions/generated/gemini/<name>.toml	generated	Gemini CLI custom command.
actions/generated/qwen/<name>.toml	generated	Qwen Code custom command.
actions/generated/codex/<name>.md	generated	OpenAI Codex CLI prompt file.
actions/generated/README.md	generated	Per-agent install-target table + count.
plugins/helix/.claude-plugin/plugin.json	manifest	The Claude Code plugin helix .
plugins/helix/commands/<name>.md	generated	Bare slash command (/<name> + / helix:<name>).
plugins/helix/commands/default-<name>.md	generated	Collision-free alias (/ default-<name>).
plugins/helix/README.md	generated	Command inventory + conflict rule (regenerated so it cannot drift).

Path	Kind	Purpose
<code>.claude-plugin/marketplace.json</code>	manifest	Local marketplace <code>helix-</code> <code>constitution</code> (plugins: <code>helix</code> , <code>scheduled-work</code>).
<code>scripts/action_prefix_lib.sh</code>	engine	The pure, agent-agnostic parse + expand library.
<code>scripts/ generate_agent_prefix_commands.sh</code>	generator	Registry → all agents' command files + both READMEs.
<code>scripts/ install_cli_agent_plugins.sh</code>	installer	Regenerate + link skills + register marketplace + install plugin.
<code>scripts/install_action_prefix.sh</code>	installer	Registers the <code>UserPromptSubmit</code> hook in <code>.claude/settings.json</code> .
<code>scripts/hooks/ action_prefix_expand.sh</code>	hook	The <code>UserPromptSubmit</code> prompt-expansion hook (Claude Code).
<code>scripts/post_update_hook.sh</code>	auto-load	On constitution pull: detects changes, calls the installer (STEP 4b).
<code>scripts/reporting/report_item.sh</code>	engine	§11.4.202 report → workable item → doc sync → tracker push.

Path	Kind	Purpose
scripts/reporting/ reporting.example.yaml	template	Consumer-owned config for the reporting engine.
skills/action-prefix-system/	skill	Agent Skill: the grammar, forms, conflict rule, unknown tokens.
skills/reporting-workable-items/	skill	Agent Skill: turning a report into a tracked item.
skills/workable-item-lifecycle/	skill	Agent Skill: Status/Type/closure/reopen/diary rules.
scripts/gates/ cm_subsystem_shortcuts.sh	gate	Sub-system shortcut gate (+ paired mutation test).
scripts/gates/ cm_covenant_114_202_propagation.sh	gate	§11.4.202 anchor-literal propagation (+ paired mutation test).
tests/action_prefix/	tests	Grammar tests, hook E2E, sub-system shortcuts, meta-test.

Generated files must never be hand-edited. Each carries a `DO NOT EDIT` banner naming its source row. Edit `actions/registry.yaml` , re-run the generator.

. Registry schema

. Top level

Key	Type	Meaning
<code>schema_version</code>	int	Currently <code>1</code> . Required — the library's <code>validate</code> refuses a registry without it.
<code>grammar</code>	map	The grammar constants (§3.2). Required — <code>grammar.prefix_regex</code> must be present for validation to pass.
<code>actions</code>	list	The registered behavioural actions (§3.3). Must be non-empty; every entry needs <code>name</code> + <code>expansion</code> .
<code>subsystems</code>	list	The curated sub-system catalogue (§3.4). Optional.

- `grammar:` `fields`

Field	Value in the shipped registry	Meaning
prefix_regex	<code>^([A-Z][A-Z0-9_]*) ::</code>	Legacy bare- <code>::</code> anchor (form 1). Kept for back-compat; its presence is what <code>apx_validate_registry</code> checks.
colon_form_regex	<code>^(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*) :: (.*)\$</code>	Forms 1 + 2. g1 = optional <code>PREFIX</code> , g2 = <code>ACTION</code> , g3 = rest.
single_colon_form_regex	<code>^(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*) : (.*)\$</code>	Form 6 (§11.4.202).
single_colon_registered_only	<code>true</code>	An unknown single-colon token is a NO-OP , never an ASK, never a sub-system.
single_colon_body_separator	<code>": "</code>	Colon + exactly one space; no space before the colon .
slash_form_regex	<code>^/(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*)\s+ (.*)\$</code>	Forms 3 + 4.

Field	Value in the shipped registry	Meaning
<code>arrow_form_regex</code>	<code>^(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*) ---> (.*)\$</code>	Form 5.
<code>arrow_body_separator</code>	<code>" ---> "</code>	One ASCII space each side of <code> ---> </code> .
<code>body_separator</code>	<code>" :: "</code>	One ASCII space each side of <code> :: </code> (forms 1/2).
<code>slash_body_separator</code>	<code>" "</code>	One space (forms 3/4).
<code>namespace_separator</code>	<code>"::"</code>	Inside the token — <code>PREFIX::ACTION</code> , no surrounding spaces .
<code>default_namespace</code>	<code>DEFAULT</code>	The reserved namespace an un-prefixed action resolves under.
<code>slash_prefix</code>	<code>"/"</code>	The slash-form leader.
<code>case_sensitive</code>	<code>true</code>	Action names are UPPERCASE-only.
<code>multiple_prefixes</code>	<code>stack</code>	<code>A :: B :: rest</code> applies A then B (outer-to-inner).
<code>escape</code>	<code>\</code>	A leading backslash makes the prefix literal.

. `actions[]` `fields`

Field	Required	Meaning
<code>name</code>	yes	UPPERCASE token, <code>[A-Z][A-Z0-9_]*</code> . The lowercased name becomes the slash-command filename.
<code>expansion</code>	yes	The text that REPLACES the prefix. Folded scalar (<code>>-</code>); newlines collapse to single spaces.
<code>version</code>	no	Registry-row version (informational).
<code>namespaces</code>	no	Which namespaces the action is registered under (shipped rows: <code>[DEFAULT]</code>).
<code>slash_bare</code>	no	<code>auto</code> = the bare <code>/name</code> form is honored unless a host built-in collides.
<code>slash_conflicts</code>	no	List of known colliding host slash commands. Non-empty $\Rightarrow$ the generator emits a <code>CONFLICT</code> comment in the command file and a warning row in the plugin README. <code>BUG</code> declares <code>[bug]</code> .

Field	Required	Meaning
summary	no	One-line description. Becomes the command's description: frontmatter and the README cell. Falls back to a generic string if absent.
rules	no	Behavioural rules the agent applies alongside the expansion. Read by LAYER 1 (the agent) — the generator does not embed them in the command files.
composes_with	no	Constitution anchors this action composes with (documentation).

• `subsystems[]` fields

Field	Required	Meaning
<code>name</code>	yes	Display name; the uppercased form is the canonical token (<code>Helix0TA</code> → <code>HELIX0TA</code>).
<code>url</code>	yes	Canonicalised to merge with any local checkout of the same repo.
<code>org</code>	no	Owning org (defaults to the org parsed from the URL).
<code>aliases</code>	no	Extra UPPERCASE tokens (abbreviations, snake forms) — e.g. <code>HX0TA</code> .

- **Registered actions (shipped)**

Action	Type produced	slash_conflicts	Summary
BACKGROUND	—	—	Run the rest of the prompt as a durable, subagent-driven background work stream, in parallel with the main stream, with hard physical proof.
REMINDER	—	—	Re-surface previously-scheduled, critical, status-UNCERTAIN work: verify the ACTUAL status from captured evidence FIRST, then act on the delta.
BUG	Bug	[bug]	Report a product defect → fully-populated Type=Bug workable item + full DB → docs → tracker sync.
TASK	Task	—	Report an internal workstream item → Type=Task workable item + full sync.

Action	Type produced	slash_conflicts	Summary
ISSUE	{Bug Feature Task}	—	Report anything trackable; classify into the §11.4.16 closed set, then create + sync.

4 1 1 4 1 4 0

1 1 4 2 0 2

4 1 . The grammar (§ . . +
§ . .)

. Six equivalent forms

All six resolve to the **same action, the same expansion, the same execution.**

#	Form	Example	Anchor regex
1	<code>ACTION :: <rest></code>	<code>BACKGROUND :: refactor the parser</code>	<code>^([A-Z][A-Z0-9_]*) :: (.*)\$</code>
2	<code>PREFIX::ACTION :: <rest></code>	<code>DEFAULT::BACKGROUND :: refactor the parser</code>	<code>^([A-Z][A-Z0-9_]*):: ([A-Z][A-Z0-9_]*) :: (.*)\$</code>
3	<code>/ACTION <rest></code>	<code>/task bump the pandoc pin</code>	<code>^/([A-Z][A-Z0-9_]*)\s+ (.*)\$</code>
4	<code>/PREFIX::ACTION <rest></code>	<code>/DEFAULT::BUG audio is silent on HDMI</code>	<code>^/([A-Z][A-Z0-9_]*):: ([A-Z][A-Z0-9_]*)\s+ (.*)\$</code>
5	<code>ACTION ---> <rest></code>	<code>REMINDER ---> the OTA signing key rotation</code>	<code>^([A-Z][A-Z0-9_]*) ---> (.*)\$</code>
6	<code>ACTION: <rest></code>	<code>ISSUE: subtitles render one frame late</code>	<code>^([A-Z][A-Z0-9_]*): (.*)\$</code>

Thus `BUG :: x` $\equiv$ `DEFAULT::BUG :: x` $\equiv$ `/DEFAULT::BUG x` $\equiv$ `BUG ---> x` $\equiv$ `BUG: x` (and `/bug x` — **except** that `bug` is a declared collision; see §5).

Form 6 was added by §11.4.202 and is the shape operators actually type. It is **REGISTERED-ACTION-ONLY by construction** (see §4.6).

. Anchoring

Every form is anchored to the **first non-blank line** of the prompt. A grammar-shaped token in the middle of a paragraph never matches. Leading blank lines are⁴skipped³; every line after the first non-blank line is carried through as part of the task (the “residual”).

. UPPERCASE only

The action token **and** the namespace are `[A-Z][A-Z0-9_]* . background :: x` never matches. This is `grammar.case_sensitive: true`, and it is what keeps the system quiet in ordinary prose.

⁴File⁴ and directory names stay lowercase snake_case (§11.4.29) — only the *tokens* are uppercase.

. Separator spacing – and why it is exactly this

The spacing rules are not cosmetic; each one exists to avoid a specific false-trigger:

Separator	Rule	The false-trigger it avoids
Namespace <code>::</code> inside the token	No surrounding spaces — <code>DEFAULT::BUG</code>	Keeps the namespace separator distinguishable from the body separator.
Body <code>::</code> (forms 1/2)	Exactly one ASCII space each side	C++ / Rust / PHP scoping — <code>Foo::Bar</code> , <code>std::vector</code> , <code>Self::new</code> — has no spaces, so it never matches.
Body <code>:</code> (form 6)	Colon + one space, no space before the colon	Distinguishes it from <code>::</code> . Still collides in <i>shape</i> with YAML <code>key: value</code> and prose <code>NOTE:</code> — which is exactly why form 6 is registered-action-only (§4.6).
Body <code>--></code> (form 5)	One ASCII space each side	An arrow inside prose (<code>a-->b</code> , <code>-></code>) does not match; only the exact <code>--></code> sequence after a valid leading token does.
Slash body (forms 3/4)	<code>/TOKEN</code> + one-or-more spaces	The leading <code>/</code> makes it unambiguous; a URL (<code>https://x/Y z</code>) never starts the line with <code>/</code> UPPERCASE .

A URL such as `https://example.com/API` cannot match: form 3 requires the line to **start** with `/` , and forms 1/2/5 require their spaced separators.

• Escape

A single leading backslash on the first non-blank line makes the prefix literal:

```
\BACKGROUND :: this is discussed, not executed
\BACKGROUND ---> ditto
\BACKGROUND ditto
\BUG: ditto
```

⁴ ⁶ The backslash is stripped and **no expansion happens**. This is how action names can be talked about (as this document does) without triggering them.

• Unknown grammar-shaped tokens

Form	Unknown token behaviour	Why
1, 2, 3, 4, 5	ASK (§11.4.66 / §11.4.105) — “this looks like an action prefix but is not registered; which registered action did you mean?” The closest registered name (longest shared prefix) is offered as a hint. Never invent an expansion (§11.4.6).	These shapes do not occur in ordinary prose, so an unknown one is almost certainly a typo or an unregistered directive.
6 (NAME: text)	NO-OP — the prompt is ordinary text, passed through unchanged. Never an ASK, never a sub-system lookup.	NOTE: / TODO: / WARNING: / FIXME: are ordinary English openers with this exact shape. Routing them to the clarify path would question every such sentence.

In no case is an unknown token silently expanded or silently dropped.

. Stacking

`A :: B :: rest` applies outer-to-inner, left-to-right: expand `A`, re-scan the residual, expand `B`, and the remaining text is the task. The expander recurses on the residual and nests `outer-expansion` + blank line + `inner-emitted`.

. The conflict rule (bare `/name` collisions)

A bare `/<name>` is honored **ONLY** when it does not collide with a host built-in slash command. Collisions are declared as **data** in the registry, never guessed:

```
- name: BUG
slash_bare: auto      # bare /bug honored *unless* a host command collides
slash_conflicts: [bug] # Claude Code ships a built-in /bug
```

What follows from that single data row:

Invocation	What happens
<code>/bug ...</code>	The HOST built-in wins. Claude Code resolves the bare name to its own <code>/bug</code> (report a bug to Anthropic). The plugin does not shadow it.
<code>/helix:bug ...</code>	The action. Claude Code's plugin namespacing (<code>/<plugin>:<command></code>) is the §11.4.140 form-4 <code>PREFIX::ACTION</code> escape — always unambiguous.
<code>/default-bug ...</code>	The action. A collision-free alias generated for every agent (see §6).
<code>BUG: ... / BUG :: ... / BUG ---> ...</code>	The action. No host command can affect these forms.

The generator materialises the rule mechanically:

- the colliding command file (`plugins/helix/commands/bug.md`) carries an explicit `CONFLICT` comment naming the host command and the two safe invocations;

- the plugin README row for a colliding action reads

```
| `/helix:bug` | `/bug` **COLLIDES** with host `bug` → use `/
helix:bug` or `/default-bug` | ... | .
```

The plugin never silently shadows a host command. Non-colliding actions

(`BACKGROUND` , `REMINDER` , `TASK` , `ISSUE`) get the plain row `| `/task` | `/helix:task` · `/default-task` | ... | .`

• Per-agent command mapping

Two command files are generated **per action**: the bare form and a `default-<name>` alias.

Agent	Generated at	Install location (consuming project)	Bare form (3)	Namespaced form (4)
Claude Code	plugins/ helix/ commands/ <name>.md and default- <name>.md	installed as the plugin helix	/<name>	/ helix:<name> (native plugin namespacing) and / default- <name>
Gemini CLI	actions/ generated/ gemini/ <name>.toml , default- <name>.toml	symlinked into .gemini/ commands/	/<name>	/default- <name>
Qwen Code	actions/ generated/ qwen/ <name>.toml , default- <name>.toml	symlinked into .qwen/ commands/	/<name>	/default- <name>
Codex CLI	actions/ generated/ codex/ <name>.md , default- <name>.md	symlinked into prompts/	/<name>	/default- <name>

All four are wired **automatically** by `install_cli_agent_plugins.sh` (see [ADMIN_MANUAL](#)) — the plugin for Claude Code, one symlink per command file for the other three (never a copy; never clobbering a hand-written file).

Per-agent capability differences — stated honestly (§11.4.6):

- Only **Claude Code** can express a `::` inside a command name — and it does so through plugin namespacing (`/helix:bug`), not literally. Gemini / Qwen / Codex **cannot** express `::` in a command name at all: a command's name is its filename/token. `default-<name>` is their form-4 equivalent, and it maps to the identical expansion, so `/<name>` $\equiv$ `/default-<name>` on every agent.
- Only **Claude Code** has a pre-submit prompt hook. On the other three agents the free-form forms (1, 2, 5, 6) are handled by LAYER 1 — the model reading its carrier block — not by a hook.
- Argument forwarding differs per agent and is handled by the generator: Claude Code and Codex use `$ARGUMENTS` ; Gemini and Qwen use `{{args}}` .
- The generator writes Claude Code's commands **straight into the plugin tree** (single source of truth, no copy). The other three agents' files land under `actions/generated/<agent>/` and the installer symlinks them into each agent's project-level command directory.

7

1 1 4 1 4 0

. Sub-system shortcuts (§ . . extension)

A grammar-shaped UPPERCASE token that is **not** a registered action may name an incorporated **sub-system / submodule**. It expands to a *sub-system context injection* (repository + org + where-checked-out + §11.4.28 equal-codebase / decoupling / dependency-layout + §11.4.37 fetch-first + §11.4.113 no-force-push + §11.4.183 full-constitution + anti-bluff), and the rest of the prompt is the task on that sub-system.

```
HXQA :: add a Challenge bank entry for the subtitle oracle
```

Two data sources feed one resolver (`apx_lookup_subsystem`):

Source	What it provides
(a) the registry <code>subsystems:</code> catalogue	Curated Helix-ecosystem sub-systems + human abbreviations (<code>HXOTA</code> → HelixOTA, <code>HXQA</code> → HelixQA, <code>CT00LKIT</code> → ClaudeToolkit, ...). These are Helix-org products, so they carry no project coupling — a consuming project's own submodules are never listed here.
(b) recursive <code>.gitmodules</code> discovery	Walks the invoking project's submodule graph (depth ≤ 6) from <code>\$HELIX_PROJECT_ROOT</code> , else the nearest ancestor of <code>\$PWD</code> containing a <code>.gitmodules</code> . Every submodule auto-derives alias tokens: uppercased name, camelCase-split snake form, and initials. A newly-added submodule is covered out of the box with no edit to the registry.

Resolution rules (all are §11.4.6 no-guessing safeguards):

- **A registered ACTION always wins** a token collision — the sub-system resolver drops any token that is a registered action name.
- Duplicate checkouts of the same submodule (same canonical URL) **collapse to one** sub-system.
- A token that maps to **more than one** sub-system is **DROPPED** → it falls through to the ASK path. It is never silently mis-expanded.
- Matching is UPPERCASE-exact.

Honest boundary (§11.4.6): sub-system resolution requires `python3` **and** `PyYAML` . Without them it returns no match and the token falls to ASK exactly as before. Behavioural actions are unaffected — they have an awk fallback.

. The engine – scripts/ action_prefix_lib.sh

⁸ ¹ The pure, agent-agnostic library reused by the hook, the generator, and the tests. It writes no files and mutates no global state.

. Public API

Function	Returns
<code>apx_registry_path</code>	The resolved registry path (<code>\$HELIX_ACTION_REGISTRY</code> , else <code><lib>/../actions/registry.yaml</code>).
<code>apx_validate_registry</code>	Exit 0 if the registry parses and has <code>schema_version</code> + <code>grammar.prefix_regex</code> + ≥ 1 action with <code>name</code> + <code>expansion</code> .
<code>apx_list_actions</code>	One action name per line.
<code>apx_lookup_expansion <NAME></code>	The action's expansion text; exit 1 if not registered.
<code>apx_lookup_subsystem <TOKEN></code>	A sub-system context expansion for a unique, non-action, uppercase-exact token; exit 1 otherwise.
<code>apx_parse_prefix <PROMPT></code>	TAB-separated <code>namespace<TAB>action<TAB>rest<TAB>form</code> . Presence of a grammar-shaped token — not registry membership.
<code>apx_expand_prompt <PROMPT></code>	The single expander. Emits a JSON object (§8.2).

• `apx_expand_prompt` result JSON

Field	Values
<code>matched</code>	<code>true</code> when a REGISTERED action or sub-system expanded.
<code>verdict</code>	<code>expand</code> · <code>noop</code> · <code>escape</code> · <code>ask</code>
<code>action</code>	The matched action name (or, on <code>ask</code> , the unknown token).
<code>namespace</code>	The resolved namespace — <code>DEFAULT</code> when the form carried none.
<code>form</code>	<code>colon</code> · <code>slash</code> · <code>arrow</code> · <code>single_colon</code>
<code>expansion</code>	The registered expansion text.
<code>residual</code>	The first-line remainder plus every following line.
<code>emitted</code>	The prompt the agent should act on: <code>expand</code> → <code><expansion>\n\n<residual></code> ; <code>escape</code> → the prompt with the leading backslash stripped; <code>noop</code> / <code>ask</code> → the original prompt unchanged.
<code>closest</code>	On <code>ask</code> : the closest registered action name (longest shared leading prefix; may be empty).
<code>kind</code>	<code>action</code> · <code>subsystem</code> — appended last, so the first nine fields are byte-stable for existing readers.

. Resolution order (first match wins)

1. **Registered behavioural ACTION** (`apx_lookup_expansion`) → `expand` ,
`kind=action` .
2. **Form 6 fall-through** — if the form was `single_colon` and the token was not a registered action → `noop` (§4.6). Resolution stops here; a sub-system is never resolved from a single-colon token.
3. **Sub-system shortcut** (`apx_lookup_subsystem`) → `expand` ,
`kind=subsystem` .
4. **ASK** — grammar-shaped but unknown → `ask` + `closest` .

. Dependencies and the fallback

The library prefers `python3` (+ `PyYAML`) for YAML parsing and for the anchored capture-group parse. When python is absent it falls back to an **awk** reader that targets exactly this registry's shape (folded `expansion:` `scalars` + simple `name:` keys) — honestly narrower than a general YAML parser. Both paths implement byte-identical anchored regexes and emit the identical 4-tuple, so they cannot drift (§11.4.50 determinism). The sub-system tier has **no** awk fallback (§7).

. Scripts

Script	What it does
<pre>scripts/ generate_agent_prefix_commands.sh</pre>	Reads the registry; for every action emits <pre>plugins/helix/commands/<name>.md + default-<name>.md , actions/generated/{gemini,qwen}/ <name>.toml + default-<name>.toml , actions/generated/codex/<name>.md + default-<name>.md ; regenerates plugins/helix/README.md (command inventory + conflict rows) and actions/ generated/README.md . Aborts if apx_validate_registry fails.</pre> Deterministic; safe to re-run. Companion doc: <pre>docs/scripts/ generate_agent_prefix_commands.md .</pre>
<pre>scripts/ install_cli_agent_plugins.sh</pre>	The single entry point. (1) runs the generator; (2) symlinks the Gemini / Qwen / Codex commands into <code>.gemini/commands/</code> , <code>.qwen/commands/</code> , <code>prompts/</code> — never clobbering a hand-written file; (3) chains <pre>install_action_prefix.sh</pre> to wire the <code>UserPromptSubmit</code> hook; (4) symlinks every constitution skill that has a <code>SKILL.md</code> into <code><project>/.claude/skills/<name></code> ; (5) <code>claude plugin marketplace add <const_root> + claude plugin install helix@helix-constitution</code> , then verifies via <code>claude plugin list</code> — it never trusts the tool's own success message (§11.4.6 / §11.4.200). Flags: <code>--check</code> (constitution and project side), <code>--skills-only</code> , <code>--skill NAME</code> , <code>--no-plugin</code> . Companion doc: <code>docs/scripts/install_cli_agent_plugins.md</code> .

Script	What it does
<code>scripts/install_action_prefix.sh</code>	The owner of the <code>UserPromptSubmit</code> hook entry in <code><project>/ .claude/ settings.json</code> (BY REFERENCE — the entry points at the constitution path; the hook is never copied), backs the file up first, and regenerates the commands. Genuinely idempotent (refuses to append a hook command it already finds), which is why chaining it from the installer above is safe.
<code>scripts/hooks/ action_prefix_expand.sh</code>	The Claude Code <code>UserPromptSubmit</code> hook. Reads the event JSON on stdin, expands via the library, and returns <code>hookSpecificOutput.additionalContext</code> . On no-match/escape it emits empty stdout (pass-through). On an unknown grammar-shaped token it injects a clarify note. On any internal error it FAILS OPEN (pass-through) — it never rejects or crashes a prompt.
<code>scripts/post_update_hook.sh</code>	§11.4.164 auto-propagation. On a constitution pull it diffs <code>ORIG_HEAD..HEAD</code> (falling back to <code>HEAD~1</code>), classifies changed files, and — when anything under <code>actions/ **</code> , <code>plugins/ **</code> , <code>.claude-plugin/ **</code> , or <code>skills/ **</code> changed — runs <code>install_cli_agent_plugins.sh</code> (STEP 4b). Also installs skills, merges MCP configs, installs git hooks, and <code>bash -n -validates</code> every touched script.

Script	What it does
scripts/reporting/report_item.sh	The §11.4.202 engine the BUG / TASK / ISSUE expansions call: create the item in the workable-items SQLite SSOT → regenerate every derived document → push to every configured external tracker (absent credentials ⇒ honest SKIP-with-reason, never a faked push). Carries zero project literals; every project-specific value comes from a consumer-owned config.

. The plugin ships no hook – deliberately

plugins/helix/ contains **only** .claude-plugin/plugin.json , commands/*.md and a generated README.md . It deliberately does **not** ship a UserPromptSubmit hook, because that would **double-expand** a prompt already expanded by scripts/hooks/action_prefix_expand.sh . The plugin owns the **commands**; install_action_prefix.sh owns the **prompt-expansion hook**.

The single install command chains both, so the operator runs one thing — but the **ownership** split is load-bearing and must be preserved: exactly one component may register that hook. (Verified invariant: after three consecutive installs, `grep -c action_prefix_expand .claude/settings.json` → 1 .)

. Skills

Three canonical Agent Skills (each a directory with a SKILL.md + a register.sh), wired into a project by **symlink** into <project>/.claude/skills/<name> — BY REFERENCE, never copied (§11.4.28 / §11.4.80: a copy diverges silently).

Skill	Loads when
<code>action-prefix-system</code>	A prompt begins with an UPPERCASE action token in any form; or the operator asks how the directive system works, how to add one, why a bare <code>/name</code> did not resolve, or what an unregistered grammar-shaped token should do. Also before inventing any expansion for an unknown token.
<code>reporting-workable-items</code>	Anything trackable is reported or discovered — a bug, a regression, a feature request, an internal task, “this is broken”, “we should add X” — or <code>/issue</code> <code>/bug</code> <code>/task</code> is invoked. Use it before answering a report in prose.
<code>workable-item-lifecycle</code>	Moving a tracked item through its lifecycle — start, ready-for-testing, close, reopen, obsolete, operator-blocked — or asking which Status/Type is legal, how to close a Feature vs a Bug, or why generated docs must never be hand-edited.

Honest boundary (§11.4.6). `constitution/skills/` also contains four **legacy** directories — `media-validator` , `scheduled-work-queue` , `session-sync` (lowercase `skill.md`) and `multitrack` (only a `register.sh`). They lack a `SKILL.md` , so they are **not discoverable as Agent Skills** and the installer deliberately **skips** them, reporting the count. They are not part of this system.

. Gates and tests

Artifact	Asserts
scripts/gates/ cm_subsystem_shortcuts.sh (+ _mutation_test.sh)	The sub-system shortcut mechanism is present and wired.
scripts/gates/ cm_covenant_114_202_propagation.sh (+ _mutation_test.sh)	The 11.4.202 anchor literal is present in every owned context carrier (CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md) across the consumer fleet. Exits non-zero on any carrier missing it. Constitution.md is intentionally not re-scanned (it is the anchor's own source of truth).
tests/action_prefix/ test_action_prefix_grammar.sh	The grammar — every form, escape, stacking, anchoring, unknown-token verdicts.
tests/action_prefix/ meta_test_action_prefix_grammar.sh	The paired §1.1 mutation — a broken grammar must make the grammar test FAIL.
tests/action_prefix/ test_hook_e2e.sh	The UserPromptSubmit hook end-to-end.
tests/action_prefix/ test_subsystem_shortcuts.sh	Sub-system alias derivation, ambiguity-drop, action-wins.

1 2

1 1 4 6

. Honest boundaries (§ . .)

Documented because they are true today, not because they are desirable:

1. **Six forms, not five.** §11.4.140's own anchor text and actions/
recognition_instruction.md (rev 3) still describe **five** forms. §11.4.202 added the sixth (single-colon NAME: <text>), and the sixth **is** implemented in

`actions/registry.yaml` (`single_colon_form_regex`) and in both parse paths of `scripts/action_prefix_lib.sh` . The carrier block is therefore **behind** the implementation. This reference documents the implementation.

2. **No prompt hook outside Claude Code.** The deterministic `UserPromptSubmit` expansion exists only for Claude Code. On Gemini / Qwen / Codex the free-form forms (1/2/5/6) rely on LAYER 1 — the model reading its context carrier. Only the **slash commands** are mechanically wired there.
3. **Sub-system shortcuts need `python3` + `PyYAML`** . Without them the token falls to ASK. Behavioural actions are unaffected (awk fallback).
4. **The marketplace lists two plugins.** `helix` (this system) and `scheduled-work` (a separate MCP-backed plugin that requires building a Go binary via `plugins/scheduled-work/build.sh`). The installer installs **only** `helix` .
5. **`--check` cannot resolve the plugin state without the `claude CLI`.** In that case it reports **UNKNOWN** — never “installed”, never “missing” (§11.4.201: a guard must assert the real condition or say honestly that it could not).